CSC103: PROGRAMMING FUNDAMENTALS
FINAL LAB PROJECT
STUDENT MANAGEMENT SYSTEM

Submitted by

Name	Registration ID
Sohayel Mahmud	SP26-BCS-114
Tauhidur Rahman Shishir	SP26-BCS-119
Md Ehtiyaz Tawsif Abid	SP26-BCT-062

Submitted to

Mr. Hassan Raza
Visiting Faculty
Department of Computer Science
COMSATS University Islamabad

JUNE 1, 2026

COMSATS UNIVERSITY ISLAMABAD
Park Road, Tarlai Kalan, Islamabad 45550, Pakistan

Table of Contents

.....0

Project Description.....2

Task Distribution 2

Features of System.....2

Modules and Entities 3

1. Main Menu Module 3

2. Teacher Authentication Module 3

3. Add Record Function 3

4. View Records Function 3

5. Search Record Function 3

6. Update Record Function.....3

7. Remove Record Function.....4

8. Student Login & Dashboard Module 4

9. Report Card & Grade Calculation Module.....4

10. File Management Module.....4

11. Input Validation Module.....4

Source Code 4

Screenshots of All Possible Outputs.....17

Case 1 (Main Menu): 17

Case 2 (Teacher Login): 17

Case 3 (Teacher Dashboard): 17

Case 4 (Add Student): 18

Case 5 (View All Records): 19

Case 6 (Update Records):.....19

Case 7 (Search Student):.....19

Case 8 (Removing Student): 19

Case 9 (Student login): 20

Case 10 (View Report Card or Export via Student Dashboard): 20

Conclusion 21

Project Description

The Student Management System is a console-based Java application designed to digitize and streamline the academic record-keeping process within an educational institution. The system provides a centralized platform where a teacher, acting as the administrator, can efficiently manage the complete data of enrolled students, including their **personal details**, **academic marks**, and **attendance information**. The application supports **two** distinct user roles — **Teacher** and **Student** — each equipped with a separate, secured login interface and a role-specific dashboard. The Teacher Dashboard grants full control over student records, enabling operations such as **adding, viewing, updating, searching,** and **removing entries**. Students can log in using their unique Student ID to access a personalized report card displaying their academic performance and grade. All data is persistently stored on disk through text files, ensuring no information is lost between program sessions. The system enforces rigorous input validation across all fields and handles all potential runtime exceptions, guaranteeing that the application does not crash under any user input scenario.

Task Distribution

Team Member	Registration No.	Responsibilities
Sohayel Mahmud	SP26-BCS-114	Teacher Login & Authentication, Teacher Dashboard Menu, Add Record, Remove Record, Update Record, Search Record, View all Records.
Tauhidur Rahman Shishir	SP26-BCS-119	Student Login & Dashboard, Report Card Generation & Export (roll_report.txt, e.g. 111_report.txt), Input Validation Helper Methods.
Md Ehtiyaz Tawsif Abid	SP26-BCT-062	Main Menu, Load Data from File & Save Data to File (Students.txt), Grade Calculation

Features of System

- **Dual Login System:** Separate and secured login portals for the Teacher (Admin) and the Student, each providing a role-specific dashboard with distinct operations.
- **Teacher Dashboard (Full CRUD):** The teacher can Add, View, Update, Search, and Remove student records through a structured, menu-driven interface.
- **Student Dashboard:** Students log in using their unique ID to view a detailed on-screen report card or export it permanently as a text file.
- **Automated Grade & GPA Calculation:** The system automatically computes a letter grade (A through F) and a corresponding grade point on a 4.0 scale based on the student's marks.
- **Persistent File Storage:** All student records are saved to students.txt and teacher credentials to credentials.txt in Text format, ensuring data survives between sessions.

- **Report Card File Export:** Students can export a fully formatted report card to a uniquely named file ([StudentID]_report.txt, e.g. 121_report.txt) for permanent reference.
- **Comprehensive Input Validation:** All numeric fields are validated for correct data type and acceptable range — Age (18–30), Marks (0–100), Attendance (0–100) — with descriptive error messages on violation.
- **Duplicate ID Prevention:** The system checks all existing records before adding a new entry, rejecting any Student ID that already exists in the database.
- **Exception Handling Throughout:** Every input and file operation is enclosed in try-catch blocks to gracefully handle NumberFormatException, Exception and IOException without crashing.
- **Login Attempt Limiting:** The Teacher login restricts failed attempts to a maximum of 3 before automatically returning to the main menu.
- **Formatted Tabular Record Display:** All student records are presented in a neatly bordered, column-aligned ASCII table for easy readability.
- **Array-Based In-Memory Storage:** Student data is managed using parallel arrays supporting up to 100 student records during runtime.

Modules and Entities

1. Main Menu Module

Loads student records and teacher credentials at startup. Displays Teacher Login, Student View, and Exit options in a continuous loop. All inputs are protected with exception handling.

2. Teacher Authentication Module

Verifies Teacher ID and password using stored credentials. Allows up to three login attempts before returning to the main menu. Successful login opens the Teacher Dashboard.

3. Add Record Function

Adds a new student's ID, Name, Age, Marks, and Attendance to the system. Checks for duplicate IDs and validates all inputs. Records are saved to the file immediately.

4. View Records Function

Displays all student records in a formatted table with proper alignment. Shows ID, Name, Age, Marks, and Attendance. Displays a message if no records exist.

5. Search Record Function

Searches for a student using their unique ID. Displays all student details if found. Shows a not-found message when no match exists.

6. Update Record Function

Allows modification of a student's Marks and Attendance. Verifies the Student ID before updating. Changes are saved to the file instantly.

7. Remove Record Function

Deletes a student record using the Student ID. Shifts remaining records to maintain array order. Updates the file immediately after deletion.

8. Student Login & Dashboard Module

Allows students to log in using their Student ID. Provides options to view or export their report card. Handles invalid inputs safely.

9. Report Card & Grade Calculation Module

Generates a complete report card with grades, GPA, and pass/fail status. Uses marks to calculate letter grades and grade points. Reports can be viewed or exported to a text file.

10. File Management Module

Handles loading and saving of student and teacher data. Uses text files for permanent storage. All file operations include exception handling.

11. Input Validation Module

Uses reusable methods to validate integer and decimal inputs. Ensures values are within acceptable ranges. Prevents invalid data entry throughout the system.

Source Code

```
import java.util.*;
import java.io.*;

public class BCS_114_BCS_119_BCT_062_Student_Management {

    public static Scanner input = new Scanner(System.in);

    public static final String FILE_NAME = "students.txt";
    public static final int MAX = 100;

    // data arrays
    public static int[] studentIDs = new int[MAX];
    public static String[] studentNames = new String[MAX];
    public static int[] studentAges = new int[MAX];
    public static double[] studentMarks = new double[MAX];
    public static int[] studentAttendance = new int[MAX];
    public static int currentCount = 0;

    // teacher login id & pass
    public static final String CRED_FILE = "credentials.txt";
    public static String teacherID = "admin";
    public static String teacherPassword = "123";

    // teacher login validation
    public static boolean teacherLogin() {
        System.out.println("\n--- Teacher Login ---");
```

```
        int attempts = 3;

        while (attempts > 0) {
            System.out.print("Enter teacher id: ");
            String idInput = input.nextLine();
            System.out.print("Enter password: ");
            String passInput = input.nextLine();

            // validate credentials
            if (idInput.equals(teacherID) && passInput.equals(teacherPassword)) {
                System.out.println("Login successful!");
                return true;
            } else {
                attempts--;
                System.out.println("Invalid ID or Password! Attempts remaining: " + attempts);
            }
        }
        System.out.println("Login failed! Returning to main menu.");
        return false;
    }

    // teacher login
    public static void teacherMenu() {
        int op = -1;
        do {
            System.out.println("\n====================================");
            System.out.println("--- Welcome to Teacher Dashboard ---");
            System.out.println("====================================");
            System.out.println("1. Add Student Record");
            System.out.println("2. View All Records");
            System.out.println("3. Update Student Record");
            System.out.println("4. Search Student Record");
            System.out.println("5. Remove Student Record");
            System.out.println("0. Log Out");
            System.out.print("Select operation: ");

            try {
                op = Integer.parseInt(input.nextLine());
                switch (op) {
                    case 1:
                        addRecord();
                        break;
                    case 2:
                        viewRecords();
                        break;
                    case 3:
                        updateRecord();
                        break;
                    case 4:
                        searchRecord();
```

```
                    break;
                case 5:
                    removeStudent();
                    break;
                case 0:
                    System.out.println("Exiting Teacher Dashboard...");
                    break;
                default:
                    System.out.println("Invalid operation! Try again.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid input! Please enter a number.");
            op = -1; // reset to keep loop running
        }
    } while (op != 0);
}

// valid input
public static int getValidInt(String prompt, int min, int max) {
    while (true) {
        try {
            System.out.print(prompt);
            int val = Integer.parseInt(input.nextLine());
            if (val < min || val > max) {
                System.out.println("Invalid range! Must be between " + min + " and " + max + ".");
                continue;
            }
            return val;
        } catch (NumberFormatException e) {
            System.out.println("Invalid input! Please enter a valid number.");
        }
    }
}

public static double getValidDouble(String prompt, double min, double max) {
    while (true) {
        try {
            System.out.print(prompt);
            double val = Double.parseDouble(input.nextLine());
            if (val < min || val > max) {
                System.out.println("Invalid range! Must be between " + min + " and " + max + ".");
                continue;
            }
            return val;
        } catch (NumberFormatException e) {
            System.out.println("Invalid input! Please enter a valid decimal number.");
        }
    }
}
```

```java
// add record
public static void addRecord() {
    System.out.println("\n--- Add New Student Record ---");
    if (currentCount >= MAX) {
        System.out.println("Database is full! Cannot add more students.");
        return;
    }

    // ID input & validation
    int id = 0;
    while (true) {
        try {
            System.out.print("Enter Student ID: ");
            id = Integer.parseInt(input.nextLine());

            // duplicate ID check
            boolean exists = false;
            for (int i = 0; i < currentCount; i++) {
                if (studentIDs[i] == id) {
                    exists = true;
                    break;
                }
            }
            if (exists) {
                System.out.println("This ID already exists! Try a unique ID.");
                continue;
            }
            break;
        } catch (NumberFormatException e) {
            System.out.println("Invalid ID! Please enter an integer number.");
        }
    }

    // name input
    System.out.print("Enter Student Name: ");
    String name = input.nextLine().replace(",", " ");

    // age input & validation
    int age = getValidInt("Enter Age (18-30): ", 18, 30);
    int attendance = getValidInt("Enter Attendance Percentage (0-100): ", 0, 100);
    double marks = getValidDouble("Enter Marks (0-100): ", 0, 100);

    // save data
    studentIDs[currentCount] = id;
    studentNames[currentCount] = name;
    studentAges[currentCount] = age;
    studentMarks[currentCount] = marks;
    studentAttendance[currentCount] = attendance;

    currentCount++;
```

```
        // save immediately to file
        saveDataToFile();
        System.out.println("Student record added successfully and saved to file!");
    }

    // view record
    public static void viewRecords() {
        System.out.println(
                "\n============================== ALL STUDENT RECORDS ==============================");
        if (currentCount == 0) {
            System.out.println("No records found! Add some students first.");
            return;
        }

        // table top border
        System.out.println("+------------+----------------------+------------+------------+-------------
-+");
        System.out.printf("| %-10s | %-20s | %-10s | %-10s | %-13s |\n", "ID", "Name", "Age", "Marks",
"Attendance");
        // header separator
        System.out.println("+------------+----------------------+------------+------------+-------------
-+");

        // table data
        for (int i = 0; i < currentCount; i++) {
            System.out.printf("| %-10d | %-20s | %-10d | %-10.2f | %-13s |\n", studentIDs[i],
studentNames[i],
                    studentAges[i], studentMarks[i], studentAttendance[i] + "%");
        }

        // table bottom border
        System.out.println("+------------+----------------------+------------+------------+-------------
-+");
    }

    // search student by id
    public static void searchRecord() {
        System.out.println("\n--- Search Student Record ---");
        if (currentCount == 0) {
            System.out.println("No records found! Database is empty.");
            return;
        }

        int searchid = 0;
        try {
            System.out.print("Enter student ID to search: ");
            searchid = Integer.parseInt(input.nextLine());
        } catch (NumberFormatException e) {
            System.out.println("Invalid ID! ID must be an integer.");
```

```
            return;
        }

        int foundindex = -1;
        for (int i = 0; i < currentCount; i++) {
            if (studentIDs[i] == searchid) {
                foundindex = i;
                break;
            }
        }

        if (foundindex != -1) {
            System.out.println("\nRecord found:");
            System.out.println("ID: " + studentIDs[foundindex]);
            System.out.println("Name: " + studentNames[foundindex]);
            System.out.println("Age: " + studentAges[foundindex]);
            System.out.println("Marks: " + studentMarks[foundindex]);
            System.out.println("Attendance: " + studentAttendance[foundindex] + "%");
        } else {
            System.out.println("Student with ID " + searchid + " not found.");
        }
    }

    // update marks and attendance
    // update marks and attendance
    public static void updateRecord() {
        System.out.println("\n--- Update Student Record ---");
        if (currentCount == 0) {
            System.out.println("No records found! database is empty.");
            return;
        }

        int searchid = 0;
        try {
            System.out.print("Enter student id to update: ");
            searchid = Integer.parseInt(input.nextLine());
        } catch (NumberFormatException e) {
            System.out.println("Invalid ID! ID must be an integer.");
            return;
        }

        int foundindex = -1;
        for (int i = 0; i < currentCount; i++) {
            if (studentIDs[i] == searchid) {
                foundindex = i;
                break;
            }
        }

        if (foundindex == -1) {
```

```
            System.out.println("Student id not found.");
            return;
        }

        System.out.println("Updating record for: " + studentNames[foundindex]);

        // update data
        studentAttendance[foundindex] = getValidInt("Enter Attendance Percentage (0-100): ", 0, 100);
        studentMarks[foundindex] = getValidDouble("Enter Marks (0-100): ", 0, 100);

        // saave the file
        saveDataToFile();
        System.out.println("Record updated successfully and saved to file!");
    }

    // remove student record by id
    public static void removeStudent() {
        System.out.println("\n--- Remove Student Record ---");
        if (currentCount == 0) {
            System.out.println("No records found! Database is empty.");
            return;
        }

        int searchid = 0;
        try {
            System.out.print("Enter student ID to remove: ");
            searchid = Integer.parseInt(input.nextLine());
        } catch (NumberFormatException e) {
            System.out.println("Invalid ID! ID must be an integer.");
            return;
        }

        int foundindex = -1;
        for (int i = 0; i < currentCount; i++) {
            if (studentIDs[i] == searchid) {
                foundindex = i;
                break;
            }
        }

        if (foundindex == -1) {
            System.out.println("Student ID not found.");
            return;
        }

        System.out.println("Removing record for: " + studentNames[foundindex]);

        for (int i = foundindex; i < currentCount - 1; i++) {
            studentIDs[i] = studentIDs[i + 1];
            studentNames[i] = studentNames[i + 1];
```

```
            studentAges[i] = studentAges[i + 1];
            studentMarks[i] = studentMarks[i + 1];
            studentAttendance[i] = studentAttendance[i + 1];
        }

        currentCount--;
        saveDataToFile();
        System.out.println("Student record removed successfully and saved to file!");
    }

    // load data from txt file to arrays
    public static void loadDataFromFile() {
        File file = new File(FILE_NAME);
        if (!file.exists()) {
            return; // no file found, starting fresh
        }

        try (Scanner filehandler = new Scanner(file)) {
            currentCount = 0;
            while (filehandler.hasNextLine() && currentCount < MAX) {
                String line = filehandler.nextLine();
                if (line.trim().isEmpty())
                    continue; // skip blank lines

                // split data by comma
                String[] tokens = line.split(",");
                studentIDs[currentCount] = Integer.parseInt(tokens[0]);
                studentNames[currentCount] = tokens[1];
                studentAges[currentCount] = Integer.parseInt(tokens[2]);
                studentMarks[currentCount] = Double.parseDouble(tokens[3]);
                studentAttendance[currentCount] = Integer.parseInt(tokens[4]);
                currentCount++;
            }
        } catch (Exception e) {
            System.out.println("Error loading data from file!");
        }
    }

    // save current arrays to txt file
    public static void saveDataToFile() {
        try (PrintWriter writer = new PrintWriter(new FileWriter(FILE_NAME))) {
            for (int i = 0; i < currentCount; i++) {
                // writing in csv format: id,name,age,marks,attendance
                writer.println(studentIDs[i] + "," +
                        studentNames[i] + "," +
                        studentAges[i] + "," +
                        studentMarks[i] + "," +
                        studentAttendance[i]);
            }
        } catch (IOException e) {
```

```
            System.out.println("Error saving data to file!");
        }
    }

    // load teacher credentials from file
    public static void loadCredentials() {
        File file = new File(CRED_FILE);
        if (!file.exists()) {
            saveCredentials(); // create file with default login if it doesn't exist
            return;
        }
        try (Scanner filehandler = new Scanner(file)) {
            if (filehandler.hasNextLine()) {
                String line = filehandler.nextLine();
                String[] tokens = line.split(",");
                if (tokens.length == 2) {
                    teacherID = tokens[0];
                    teacherPassword = tokens[1];
                }
            }
        } catch (Exception e) {
            System.out.println("Error loading credentials!");
        }
    }

    // save teacher credentials to file
    public static void saveCredentials() {
        try (PrintWriter writer = new PrintWriter(new FileWriter(CRED_FILE))) {
            writer.println(teacherID + "," + teacherPassword);
        } catch (IOException e) {
            System.out.println("Error saving credentials!");
        }
    }

    // student login and validation
    public static void studentMenu() {
        System.out.println("\n--- Student Login ---");
        if (currentCount == 0) {
            System.out.println("No records in the system yet.");
            return;
        }

        int searchid = 0;
        try {
            System.out.print("Enter your student ID to login: ");
            searchid = Integer.parseInt(input.nextLine());
        } catch (NumberFormatException e) {
            System.out.println("Invalid ID! ID must be an integer number.");
            return;
        }
```

```
    int foundindex = -1;
    for (int i = 0; i < currentCount; i++) {
        if (studentIDs[i] == searchid) {
            foundindex = i;
            break;
        }
    }

    if (foundindex != -1) {
        System.out.println("Login successful! WELCOME, " + studentNames[foundindex] + " !!!");
        studentDashboard(foundindex);
    } else {
        System.out.println("Student ID not found. Login failed!");
    }
}

// separate method for student operations loop
public static void studentDashboard(int index) {
    int op = -1;
    do {
        System.out.println("\n--- Student Dashboard ---");
        System.out.println("1. View Report Card");
        System.out.println("2. Export Report Card to File");
        System.out.println("0. Log Out");
        System.out.print("Select operation: ");

        try {
            op = Integer.parseInt(input.nextLine());
            switch (op) {
                case 1:
                    printReportCard(index);
                    break;
                case 2:
                    exportReportCard(index);
                    break;
                case 0:
                    System.out.println("Exiting Student Dashboard...");
                    break;
                default:
                    System.out.println("Invalid operation! Try again.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid input! Please enter a number.");
            op = -1;
        }
    } while (op != 0);
}

// grade calculator
```

```java
public static String[] calculateGrade(double m) {
    String letter = "F";
    double gradePoint = 0.0;
    String status = "FAIL";

    if (m >= 85) {
        letter = "A";
        gradePoint = 4.0;
        status = "PASS (Excellent)";
    } else if (m >= 80) {
        letter = "A-";
        gradePoint = 3.7;
        status = "PASS";
    } else if (m >= 75) {
        letter = "B+";
        gradePoint = 3.3;
        status = "PASS";
    } else if (m >= 71) {
        letter = "B";
        gradePoint = 3.0;
        status = "PASS";
    } else if (m >= 68) {
        letter = "B-";
        gradePoint = 2.7;
        status = "PASS";
    } else if (m >= 64) {
        letter = "C+";
        gradePoint = 2.3;
        status = "PASS";
    } else if (m >= 61) {
        letter = "C";
        gradePoint = 2.0;
        status = "PASS";
    } else if (m >= 58) {
        letter = "C-";
        gradePoint = 1.7;
        status = "PASS";
    } else if (m >= 50) {
        letter = "D";
        gradePoint = 1.3;
        status = "PASS (Minimum Passing)";
    } else {
        letter = "F";
        gradePoint = 0.0;
        status = "FAIL";
    }

    return new String[] { status, letter, String.valueOf(gradePoint) };
}
```

```
    // sub-method to just print report card on console
    public static void printReportCard(int index) {
        System.out.println("\n====================================");
        System.out.println("          STUDENT REPORT CARD      ");
        System.out.println("====================================");
        System.out.println("ID:         " + studentIDs[index]);
        System.out.println("Name:       " + studentNames[index]);
        System.out.println("Age:        " + studentAges[index]);
        System.out.println("Marks:      " + studentMarks[index] + " / 100");
        System.out.println("Attendance: " + studentAttendance[index] + "%");
        System.out.println("====================================");

        String[] gradeData = calculateGrade(studentMarks[index]);
        String status = gradeData[0];
        String letter = gradeData[1];
        String gradePoint = gradeData[2];

        System.out.println("Status: " + status);
        System.out.println("Letter Grade: " + letter + "    Grade Point: " + gradePoint);
        System.out.println("====================================");
    }

    // sub-method to export report card to text file
    public static void exportReportCard(int index) {
        String filename = studentIDs[index] + "_report.txt";
        try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
            writer.println("====================================");
            writer.println("         STUDENT REPORT CARD      ");
            writer.println("====================================");
            writer.println("ID:         " + studentIDs[index]);
            writer.println("Name:       " + studentNames[index]);
            writer.println("Age:        " + studentAges[index]);
            writer.println("Marks:      " + studentMarks[index] + " / 100");
            writer.println("Attendance: " + studentAttendance[index] + "%");
            writer.println("====================================");

            String[] gradeData = calculateGrade(studentMarks[index]);
            String status = gradeData[0];
            String letter = gradeData[1];
            String gradePoint = gradeData[2];

            writer.println("Status: " + status);
            writer.println("Letter Grade: " + letter + "    Grade Point: " + gradePoint);
            writer.println("====================================");
            System.out.println("Report exported successfully to " + filename);
        } catch (IOException e) {
            System.out.println("Error exporting report card!");
        }
    }
```

```
    public static void main(String[] args) throws IOException {
        loadDataFromFile();
        loadCredentials();

        int identity = 0;
        boolean keepRunning = true;

        while (keepRunning) {
            try {
                System.out.println("\n=== STUDENT MANAGEMENT SYSTEM ===");
                System.out.println("1. Teacher Login");
                System.out.println("2. Student View");
                System.out.println("0. Exit Program");
                System.out.print("Enter identity: ");
                identity = Integer.parseInt(input.nextLine());

                switch (identity) {
                    case 1:
                        if (teacherLogin()) {
                            teacherMenu();
                        }
                        break;
                    case 2:
                        studentMenu();
                        break;
                    case 0:
                        keepRunning = false;
                        break;
                    default:
                        System.out.println("Invalid identity! Please enter 1, 2, or 0.");
                }
            } catch (NumberFormatException e) {
                System.out.println("Invalid input! Please enter a number.");
            }
        }

        saveDataToFile();
        System.out.println("Data saved successfully. Goodbye!");
    }
}
```

Screenshots of All Possible Outputs

Case 1 (Main Menu):

```
=== STUDENT MANAGEMENT SYSTEM ===
1. Teacher Login
2. Student View
0. Exit Program
Enter identity:
```

Figure 1 Home Menu for Selecting Identity

Case 2 (Teacher Login):

```
--- Teacher Login ---
Enter teacher id: sohayel
Enter password: 1234
Invalid ID or Password! Attempts remaining: 2
Enter teacher id: mahmud
Enter password: 4321
Invalid ID or Password! Attempts remaining: 1
Enter teacher id: abid
Enter password: 9876
Invalid ID or Password! Attempts remaining: 0
Login failed! Returning to main menu.
```

Figure 2 Failed Login Attempted for Teachers

```
--- Teacher Login ---
Enter teacher id: admin
Enter password: 123
Login successful!
```

Figure 3 Login Successful for Teachers

Case 3 (Teacher Dashboard):

```
====================================
--- Welcome to Teacher Dashboard ---
====================================
1. Add Student Record
2. View All Records
3. Update Student Record
4. Search Student Record
5. Remove Student Record
0. Log Out
Select operation:
```

Figure 2 Teachers Dashboard

Case 4 (Add Student):

```
--- Add New Student Record ---
Enter Student ID: 152
Enter Student Name: Jim
Enter Age (18-30): 22
Enter Attendance Percentage (0-100): 99
Enter Marks (0-100): 99
Student record added successfully and saved to file!
```

Figure 5 Successfully Added a student

```
--- Add New Student Record ---
Enter Student ID: 101
This ID already exists! Try a unique ID.
Enter Student ID:
```

Figure 6 Failed Adding for Duplicate ID

```
--- Add New Student Record ---
Enter Student ID: 153
Enter Student Name: Jim
Enter Age (18-30): 14
Invalid range! Must be between 18 and 30.
Enter Age (18-30):
```

Figure 7 Outrange for Age

```
--- Add New Student Record ---
Enter Student ID: 153
Enter Student Name: Jim
Enter Age (18-30): 14
Invalid range! Must be between 18 and 30.
Enter Age (18-30): 23
Enter Attendance Percentage (0-100): 103
Invalid range! Must be between 0 and 100.
Enter Attendance Percentage (0-100):
```

Figure 8 Outrange for Attendance

```
--- Add New Student Record ---
Enter Student ID: 153
Enter Student Name: Jim
Enter Age (18-30): 14
Invalid range! Must be between 18 and 30.
Enter Age (18-30): 23
Enter Attendance Percentage (0-100): 103
Invalid range! Must be between 0 and 100.
Enter Attendance Percentage (0-100): 88
Enter Marks (0-100): 234
Invalid range! Must be between 0.0 and 100.0.
Enter Marks (0-100):
```

Figure 9 Outrange for Marks

```
--- Add New Student Record ---
Enter Student ID: 154
Enter Student Name: Kamrul
Enter Age (18-30): asdf
Invalid input! Please enter a valid number.
Enter Age (18-30):
```

Figure 10 Invalid type for Age

```
--- Add New Student Record ---
Enter Student ID: 154
Enter Student Name: Kamrul
Enter Age (18-30): asdf
Invalid input! Please enter a valid number.
Enter Age (18-30): 26
Enter Attendance Percentage (0-100): asdf
Invalid input! Please enter a valid number.
Enter Attendance Percentage (0-100):
```

Figure 11 Invalid type for Attendance

```
--- Add New Student Record ---
Enter Student ID: 154
Enter Student Name: Kamrul
Enter Age (18-30): asdf
Invalid input! Please enter a valid number.
Enter Age (18-30): 26
Enter Attendance Percentage (0-100): asdf
Invalid input! Please enter a valid number.
Enter Attendance Percentage (0-100): 100
Enter Marks (0-100): qwer
Invalid input! Please enter a valid decimal number.
Enter Marks (0-100):
```

Figure 12 Invalid type of Marks

Case 5 (View All Records):

```
Select operation: 2

============================ ALL STUDENT RECORDS ============================
+------------+----------------------+------------+------------+---------------+
| ID         | Name                 | Age        | Marks      | Attendance    |
+------------+----------------------+------------+------------+---------------+
| 101        | Sohayel Mahmud       | 21         | 92.50      | 88%           |
| 102        | Abdullah Al Rifat    | 22         | 85.00      | 95%           |
| 103        | Naimur Rahman        | 20         | 78.20      | 84%           |
| 104        | Afsana Mimi          | 19         | 65.40      | 72%           |
| 105        | Tanvir Ahmed         | 23         | 45.00      | 90%           |
| 106        | Sajid Islam          | 21         | 89.50      | 81%           |
| 107        | Fariha Zaman         | 22         | 91.20      | 93%           |
| 108        | Mahfuz Anam          | 20         | 55.60      | 68%           |
| 109        | Anika Tabassum       | 19         | 73.80      | 85%           |
| 110        | Asif Entaz           | 24         | 38.00      | 60%           |
| 111        | eee                  | 23         | 23.00      | 23%           |
| 112        | Rifat Khan           | 21         | 82.40      | 87%           |
| 113        | Sadia Sultana        | 20         | 69.50      | 79%           |
| 114        | Imran Hossen         | 22         | 48.50      | 74%           |
| 115        | Nusrat Jahan         | 19         | 95.00      | 96%           |
| 116        | Arif Rayhan          | 23         | 76.30      | 83%           |
| 117        | Jannatul Ferdous     | 21         | 62.00      | 70%           |
| 118        | Kazi Anas            | 20         | 88.70      | 91%           |
```

Figure 13 View All the Students Data

Case 6 (Update Records):

```
--- Update Student Record ---
Enter student id to update: 153
Updating record for: Jim
Enter Attendance Percentage (0-100): 89
Enter Marks (0-100): 99
Record updated successfully and saved to file!
```

Figure 14 Successfully Updated Student Data

```
--- Update Student Record ---
Enter student id to update: 199
Student id not found.
```

Figure 15 Invalid Student ID for Update

Case 7 (Search Student):

```
--- Search Student Record ---
Enter student ID to search: 139

Record found:
ID: 139
Name: Mastura Rahman Ifrit
Age: 20
Marks: 100.0
Attendance: 100%
```

Figure 16 Successfully Searching Student Data

```
--- Search Student Record ---
Enter student ID to search: 199
Student with ID 199 not found.
```

Figure 17 Student ID not Found

```
--- Search Student Record ---
Enter student ID to search: asdf
Invalid ID! ID must be an integer.
```

Figure 18 ID not Found for Removing Student

Case 8 (Removing Student):

```
--- Remove Student Record ---
Enter student ID to remove: 154
Removing record for: Kamrul
Student record removed successfully and saved to file!
```

Figure 19 Removing Student Successfully

```
--- Remove Student Record ---
Enter student ID to remove: 199
Student ID not found.
```

Figure 20 Invalid Input type for Searching Student

```
--- Remove Student Record ---
Enter student ID to remove: jim
Invalid ID! ID must be an integer.
```

Figure 21 Invalid Input Type for Removing Student

Case 9 (Student login):

```
--- Student Login ---
Enter your student ID to login: 139
Login successful! WELCOME, Mastura Rahman Ifrit !!!
```

Figure 22 Successfully Student Login

```
--- Student Login ---
Enter your student ID to login: 199
Student ID not found. Login failed!
```

Figure 23 Failed Student Login

Case 10 (View Report Card or Export via Student Dashboard):

```
--- Student Dashboard ---
1. View Report Card
2. Export Report Card to File
0. Log Out
Select operation:
```

Figure 24 Student Dashboard

```
Select operation: 2
Report exported successfully to 139_report.txt
```

139_report.txt

```
1   ====================================
2           STUDENT REPORT CARD
3   ====================================
4   ID:          139
5   Name:        Mastura Rahman Ifrit
6   Age:         20
7   Marks:       100.0 / 100
8   Attendance: 100%
9   ====================================
10  Status: PASS (Excellent)
11  Letter Grade: A    Grade Point: 4.0
12  ====================================
13
```

Figure 25 Export Report in a .txt File

```
Select operation: 1

====================================
          STUDENT REPORT CARD
====================================
ID:          139
Name:        Mastura Rahman Ifrit
Age:         20
Marks:       100.0 / 100
Attendance: 100%
====================================
Status: PASS (Excellent)
Letter Grade: A    Grade Point: 4.0
====================================

--- Student Dashboard ---
```

Figure 26 Print Report on Terminal

Conclusion

The Student Management System successfully fulfills all the objectives set out for this project by delivering a fully functional, menu-driven console application built in Java. The system effectively implements a dual-role login mechanism, granting the Teacher administrative control over all student records while providing Students with secure, read-only access to their personal academic data. All core operations — Add, View, Update, Search, and Remove — have been implemented with robust input validation and exception handling, ensuring the application remains stable and crash-free under all user interactions. The use of parallel arrays demonstrates a solid understanding of array-based data management in Java, while file handling through text-based storage ensures that all records persist permanently on disk between program sessions. The automated grade calculation and report card export features add meaningful academic utility to the system beyond basic record management. Overall, this project provided valuable hands-on experience in applying core programming fundamentals — including arrays, file I/O, exception handling, input validation, and modular method design — in the context of a real-world, team-developed application.